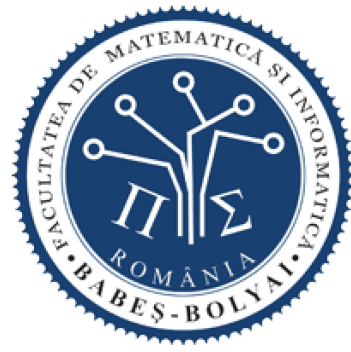




Operations Research

13 – Optimization algorithms

Mihai Suciu

Faculty of Mathematics and Computer Science

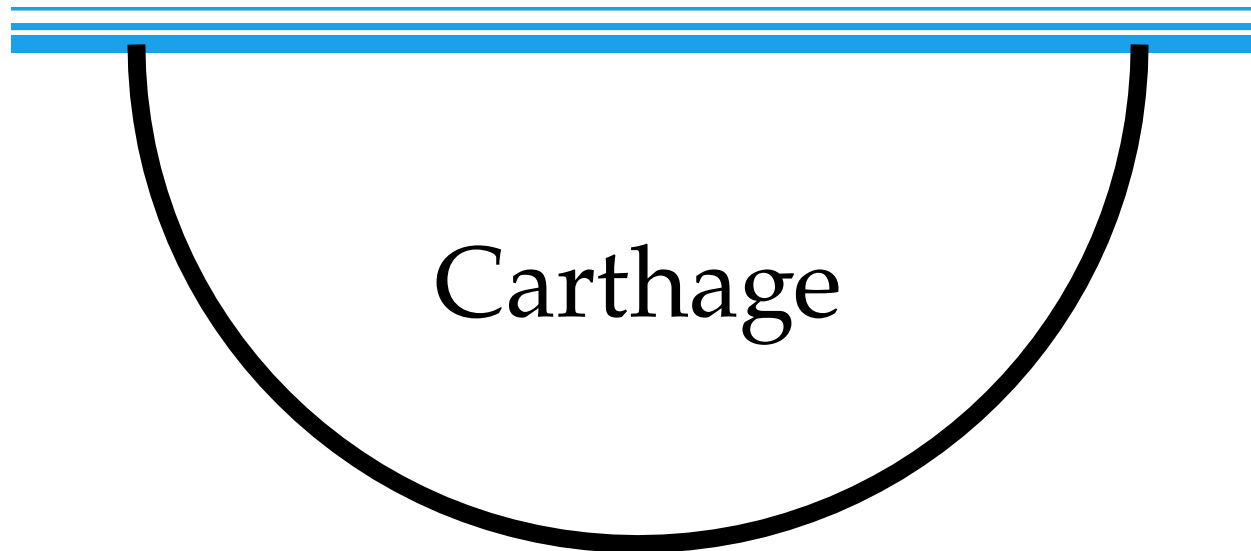
Introduction

- Applications of Optimization
 - Physics
 - Business
 - Biology
 - Engineering
- Objectives to Optimize
 - Efficiency
 - Safety
 - Accuracy

- Constraints
 - Cost
 - Weight
 - Structural Integrity
- Challenges
 - High-Dimensional Search Spaces
 - Multiple Competing Objectives
 - Model Uncertainty

A History

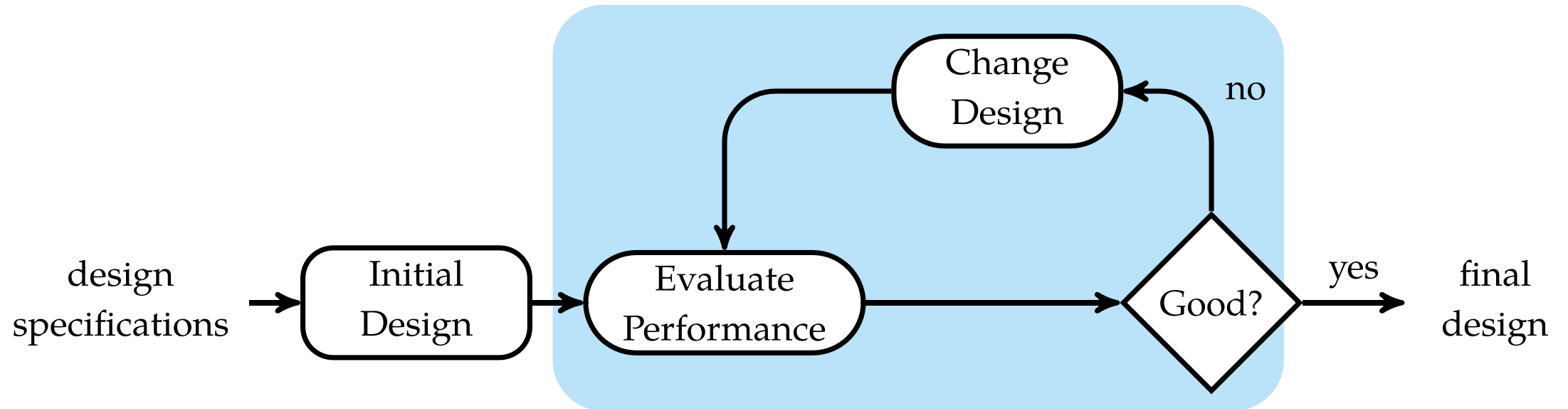
- Queen Dido's Optimization Problem
- sea



A History

- Calculus
- Numerical Algorithms
- Artificial Intelligence

Optimization Process



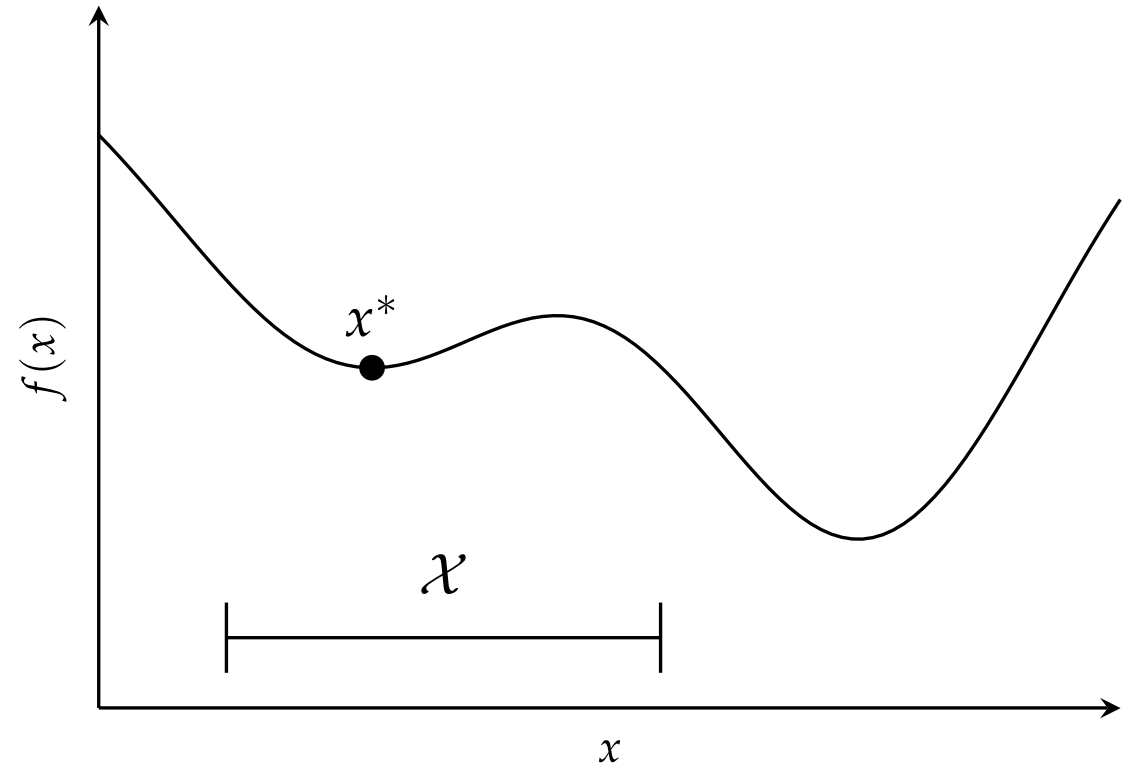
Basic Optimization Problem

$$\begin{array}{ll}\underset{\mathbf{x}}{\text{minimize}} & f(\mathbf{x}) \\ \text{subject to} & \mathbf{x} \in \mathcal{X}\end{array}$$

- Design Point
- Design Variables
- Objective Function
- Feasible Set
- Minimizer

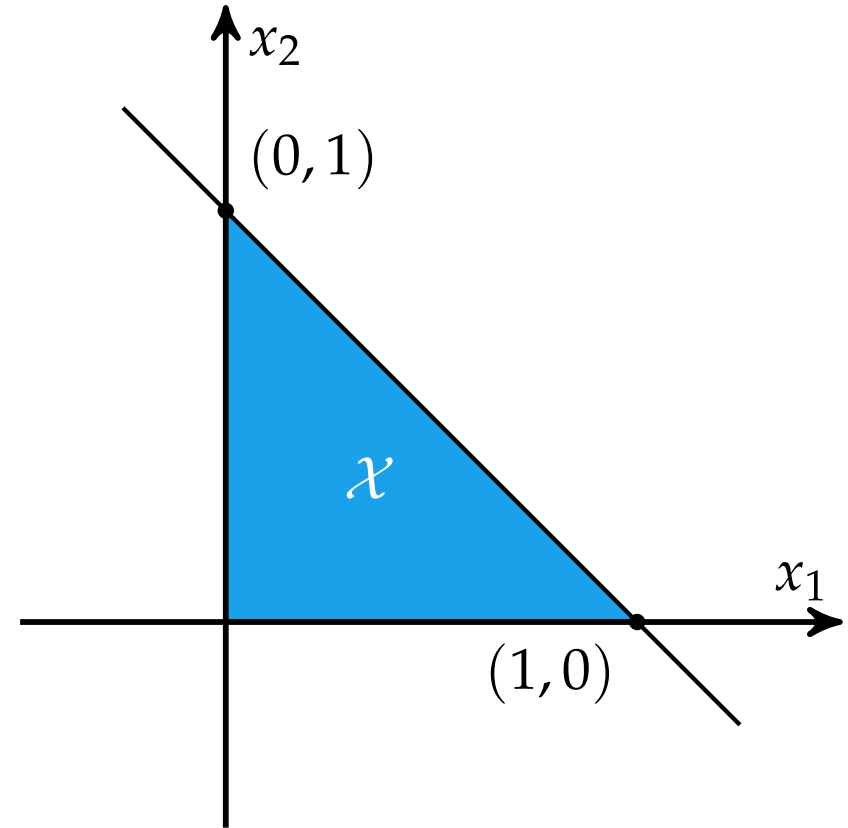
Basic Optimization Problem

$$\begin{array}{ll}\underset{\mathbf{x}}{\text{minimize}} & f(\mathbf{x}) \\ \text{subject to} & \mathbf{x} \in \mathcal{X}\end{array}$$



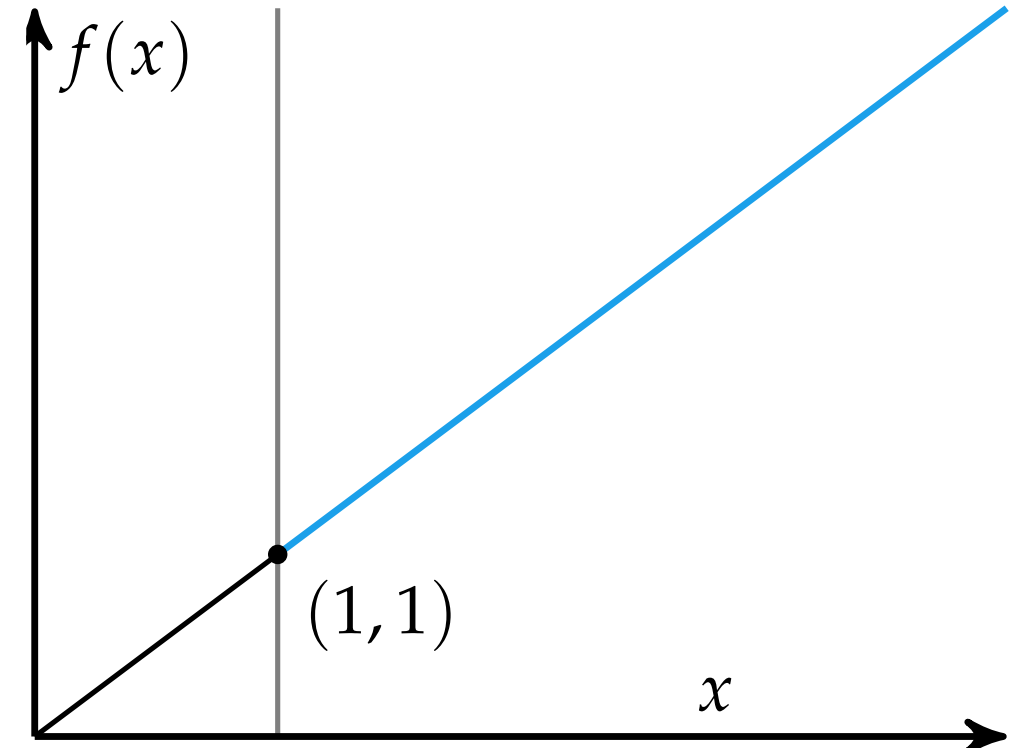
Constraints

$$\begin{array}{ll}\text{minimize} & f(x_1, x_2) \\ \text{subject to} & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_1 + x_2 \leq 1\end{array}$$



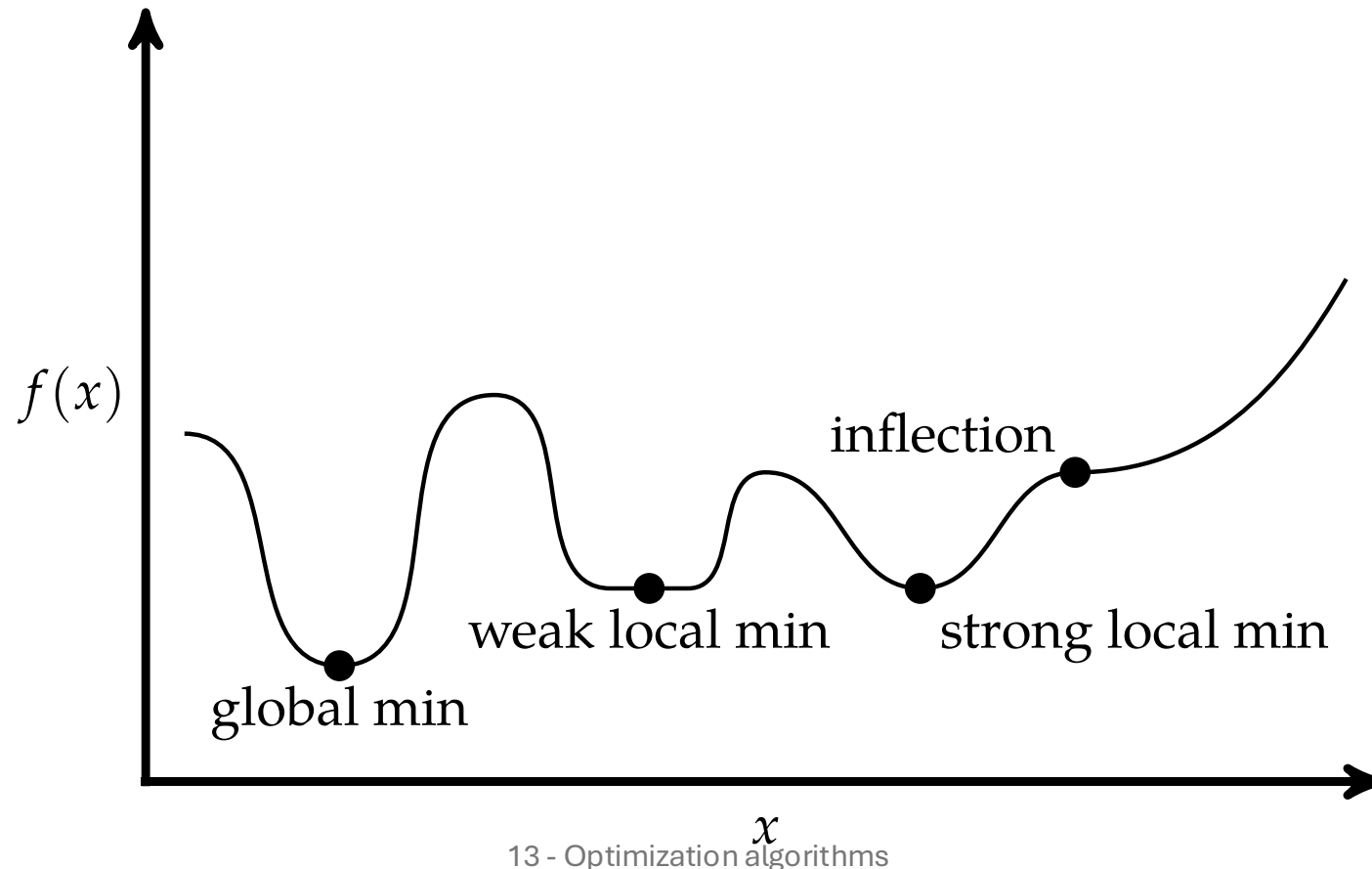
Constraints

$$\begin{array}{ll}\underset{x}{\text{minimize}} & x \\ \text{subject to} & x > 1\end{array}$$



Critical Points

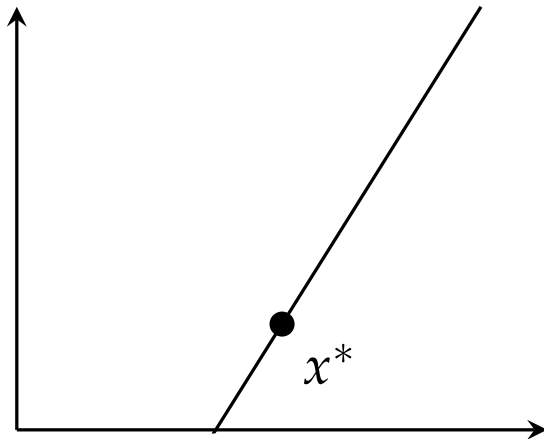
- Univariate Function



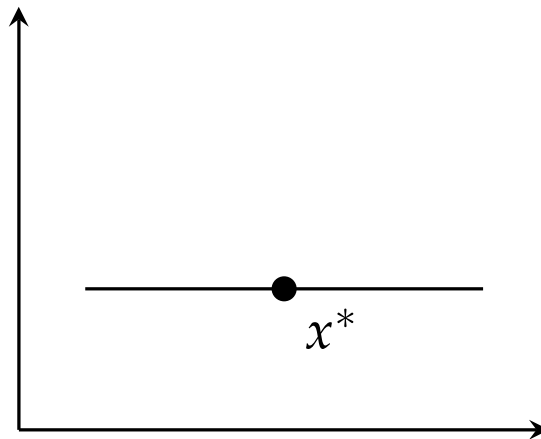
Conditions for Local Minima

- *Univariate Function*

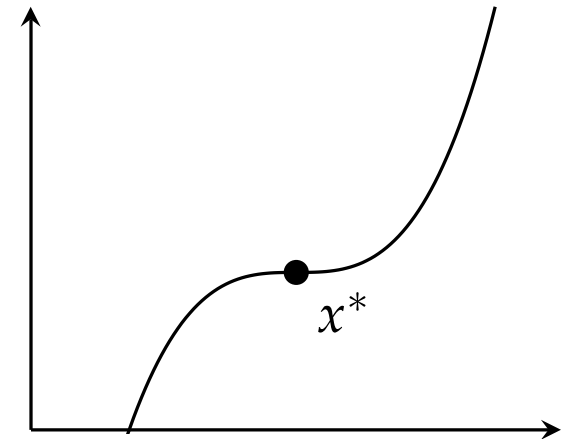
1. $f'(x^*) = 0$, the first-order necessary condition (FONC)
2. $f''(x^*) \geq 0$, the second-order necessary condition (SONC)



SONC but not FONC



FONC and SONC



FONC and SONC

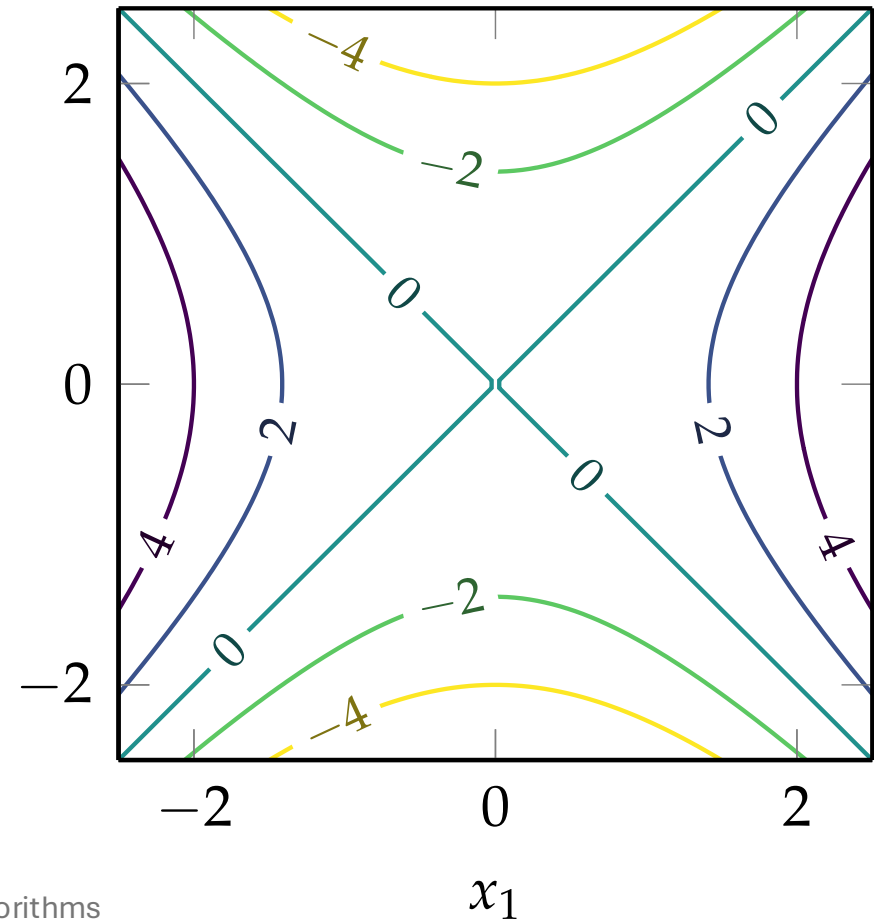
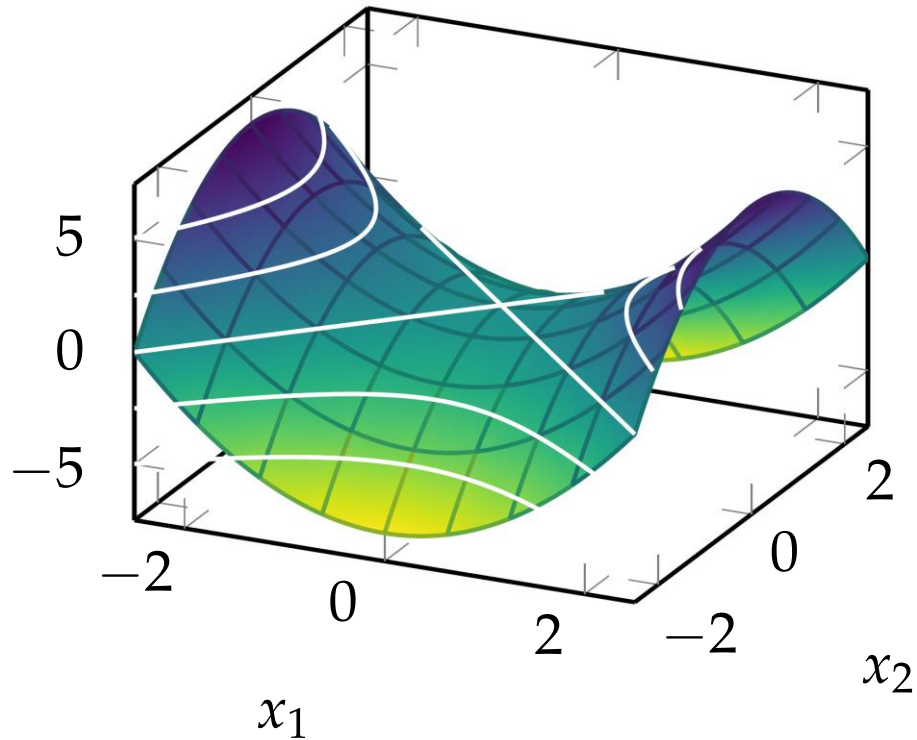
Conditions for Local Minima

- *Multivariate Functions*

1. $\nabla f(\mathbf{x}^*) = 0$, the first-order necessary condition (FONC)
2. $\nabla^2 f(\mathbf{x}^*) \geq 0$, the second-order necessary condition (SONC)

Contour Plots

- $f(x_1, x_2) = x_1^2 - x_2^2$



Population Methods

Population Methods

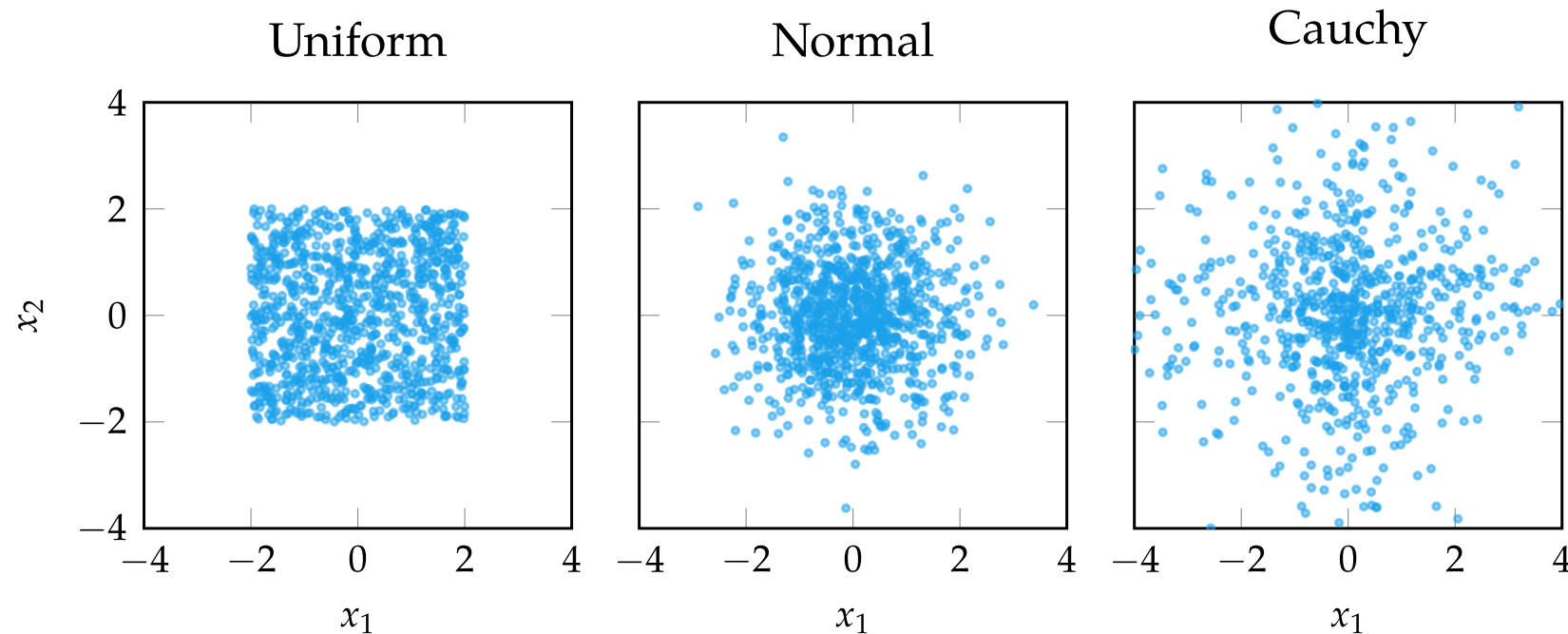
- Instead of optimizing a single design point, population methods optimize a collection of individuals
- Many individuals prevents algorithm from being stuck in a local minimum
- Useful information can be shared between individuals
- Stochastic in nature
- Easy to parallelize

```
abstract type PopulationMethod end

function population_method(M::PopulationMethod, f, designs, k_max)
    population = init!(M, f, designs)
    for k in 1:k_max
        population = step!(M, f, population)
    end
    return population
end
```

Initialization

- Population methods begin with an initial population
- Common initializations are uniform, normal distribution, and Cauchy distribution



Genetic Algorithms

- Inspired by biological evolution where the fittest individuals pass their genetic information to the next generation
- Individuals are interpreted as chromosomes
- The fittest individuals are determined by selection
- The next generation is formed by selecting the fittest individuals and performing crossover and mutation

```

struct GeneticAlgorithm <: PopulationMethod
    S # SelectionMethod
    C # CrossoverMethod
    U # MutationMethod
end

init!(M::GeneticAlgorithm, f, designs) = designs

function step!(M::GeneticAlgorithm, f, population)
    S, C, U = M.S, M.C, M.U
    parents = select(S, f.(population))
    children = [crossover(C,population[p[1]],population[p[2]])
                for p in parents]
    return [mutate(U, c) for c in children]
end

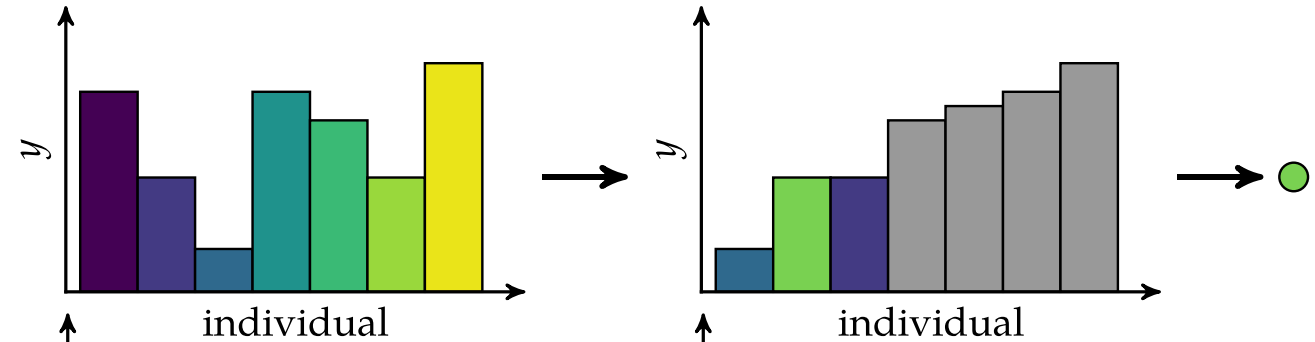
```

Genetic Algorithms: Selection

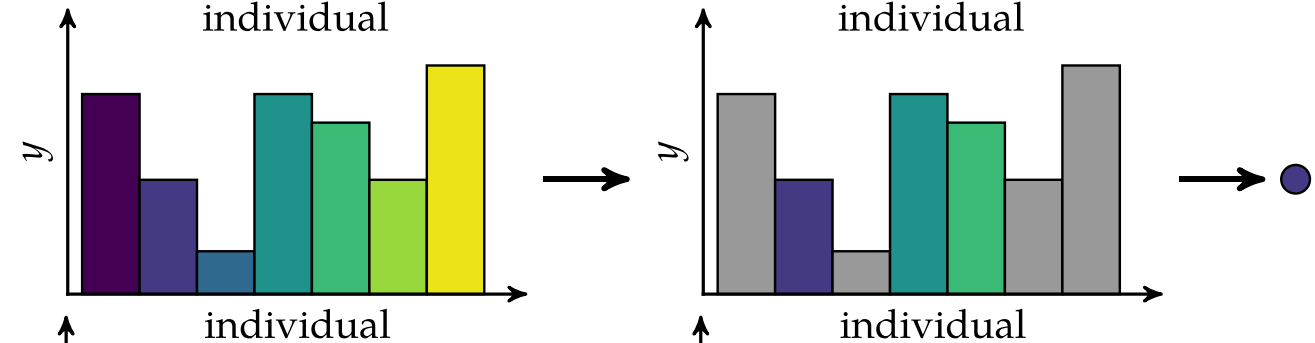
- Determining which individuals pass their genetic information on to the next generation
- Truncation selection: truncate the lowest performers
- Tournament selection: selects fittest out of k randomly chosen individuals
- Roulette Wheel selection: individuals are chosen with probability proportional to their fitness

Genetic Algorithms: Selection

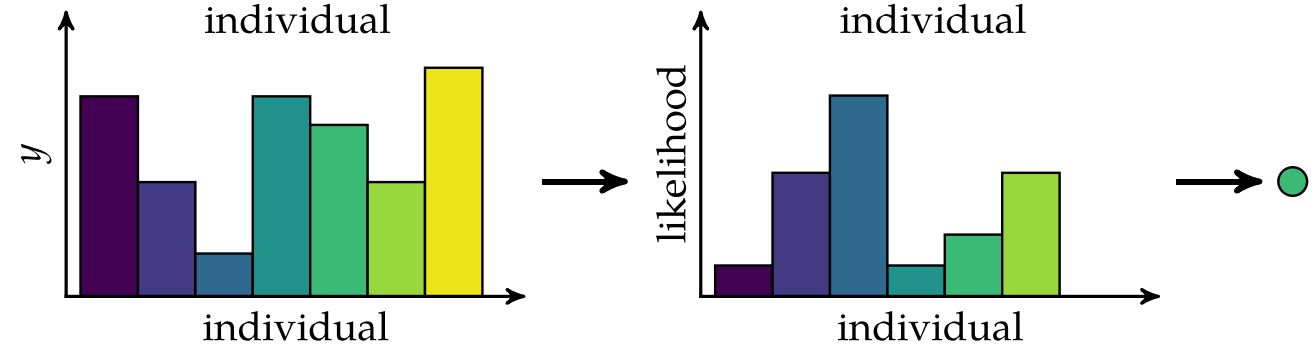
- Truncation Selection



- Tournament Selection



- Roulette Wheel Selection



```
abstract type SelectionMethod end
```

```
# Pick pairs randomly from top k parents  
struct TruncationSelection <: SelectionMethod  
    k # top k to keep  
end
```

```
function select(t::TruncationSelection, y)  
    p = sortperm(y)  
    return [p[rand(1:t.k, 2)] for i in y]  
end
```

```
# Pick parents by choosing best among random subsets  
struct TournamentSelection <: SelectionMethod  
    k # top k to keep  
end
```

```
function select(t::TournamentSelection, y)  
    getparent() = begin  
        p = randperm(length(y))  
        p[argmin(y[p[1:t.k]])]  
    end  
    return [[getparent(), getparent()] for i in y]  
end
```

```
# Sample parents proportionately to fitness  
struct RouletteWheelSelection <: SelectionMethod end
```

```
function select(::RouletteWheelSelection, y)  
    y = maximum(y) .- y  
    cat = Categorical(normalize(y, 1))  
    return [rand(cat, 2) for i in y]  
end
```

Genetic Algorithms: Chromosomes

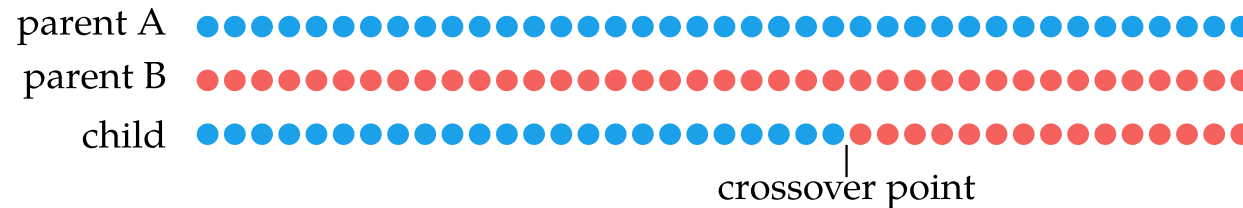
- Simplest representation is the binary string chromosome



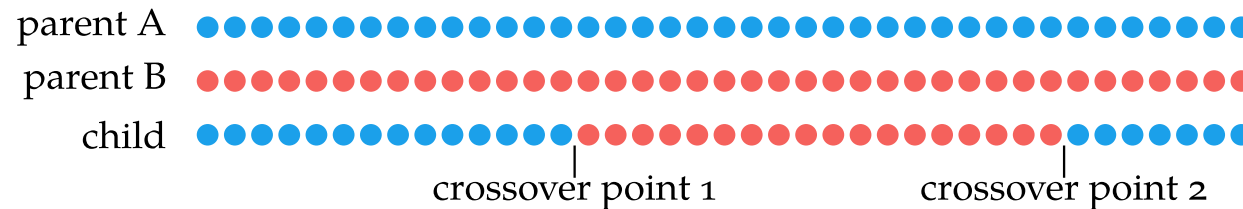
- Chromosomes are more commonly represented as real-valued chromosomes which are simply real-valued vectors
- Typically initialized randomly

Genetic Algorithms: Crossover

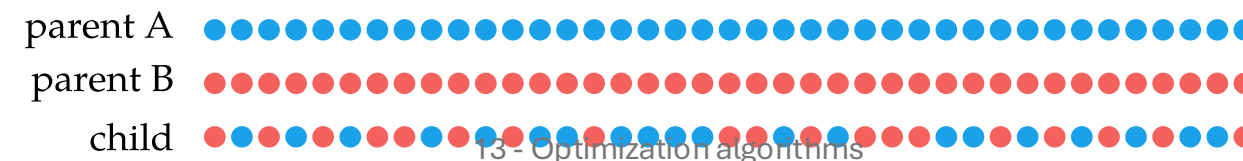
- Combines the chromosomes of the parents to form children
- Single-point crossover: swap occurs after single crossover point



- Two-point crossover: two crossover points



- Uniform crossover: each bit has 50% chance of crossover



```

abstract type CrossoverMethod end

struct SinglePointCrossover <: CrossoverMethod end

function crossover(::SinglePointCrossover, a, b)
    i = rand(eachindex(a))
    return [a[1:i]; b[i+1:end]]
end

struct TwoPointCrossover <: CrossoverMethod end

function crossover(::TwoPointCrossover, a, b)
    n = length(a)
    i, j = rand(1:n, 2)
    if i > j
        (i,j) = (j,i)
    end
    return [a[1:i]; b[i+1:j]; a[j+1:n]]
end

struct UniformCrossover <: CrossoverMethod
    p # crossover probability
end

function crossover(U::UniformCrossover, a, b)
    return [rand() > U.p ? u : v for (u,v) in zip(a,b)]
end

struct InterpolationCrossover <: CrossoverMethod
    λ # interpolant
end

crossover(C::InterpolationCrossover, a, b) = (1-C.λ)*a + C.λ*b

```

Genetic Algorithms: Mutation

- Mutation supports exploration of new areas of design space
- Each bit or real-valued element has a probability of being flipped or modified by noise
- The probability of an element mutating is called mutation rate

before mutation 

after mutation 

```

abstract type MutationMethod end

struct DistributionMutation <: MutationMethod
     $\lambda$  # mutation rate
    D # mutation distribution
end

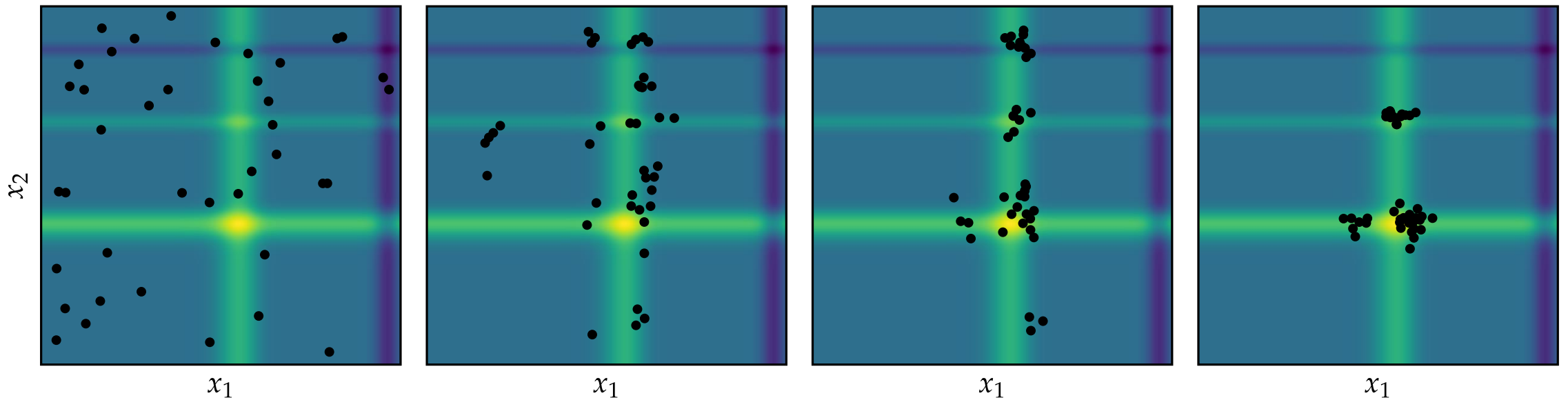
function mutate(M::DistributionMutation, child)
    return [rand() < M. $\lambda$  ? v + rand(M.D) : v for v in child]
end

GaussianMutation( $\sigma$ ) = DistributionMutation(1.0, Normal(0, $\sigma$ ))

```

Genetic Algorithms

- Genetic algorithm with truncation selection, single point crossover, and Gaussian mutation applied to Michalewicz function



```

import Random: seed!
import LinearAlgebra: norm
seed!(0) # set random seed for reproducible results
f = x→norm(x)
m = 100 # population size
k_max = 10 # number of iterations
function rand_population_uniform(m, a, b)
    d = length(a)
    return [a+rand(d).*(b-a) for i in 1:m]
end
population = rand_population_uniform(m, [-3,3], [3,3])
M = GeneticAlgorithm(
    TruncationSelection(10), # select top 10
    SinglePointCrossover(),
    GaussianMutation(0.5)) # perturb with 0.5 standard deviation
population = population_method(M, f, population, k_max)
x = argmin(f, population)
@show x

x = [-0.035070453820023516, 0.03762926957036672]

```

Differential Evolution

- Improves each individual $\mathbf{x}$ by recombining other individuals according to a simple formula
 1. Choose three random, distinct individuals $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$
 2. Construct interim design $\mathbf{z} = \mathbf{a} + w(\mathbf{b} - \mathbf{c})$
 3. Choose a random dimension to optimize in
 4. Construct candidate $\mathbf{x}'$ via binary crossover of $\mathbf{x}$ and $\mathbf{z}$
 5. Insert better design between $\mathbf{x}$ and $\mathbf{x}'$ into next generation

```

mutable struct DifferentialEvolution <: PopulationMethod
    p # crossover probability
    w # differential weight
end

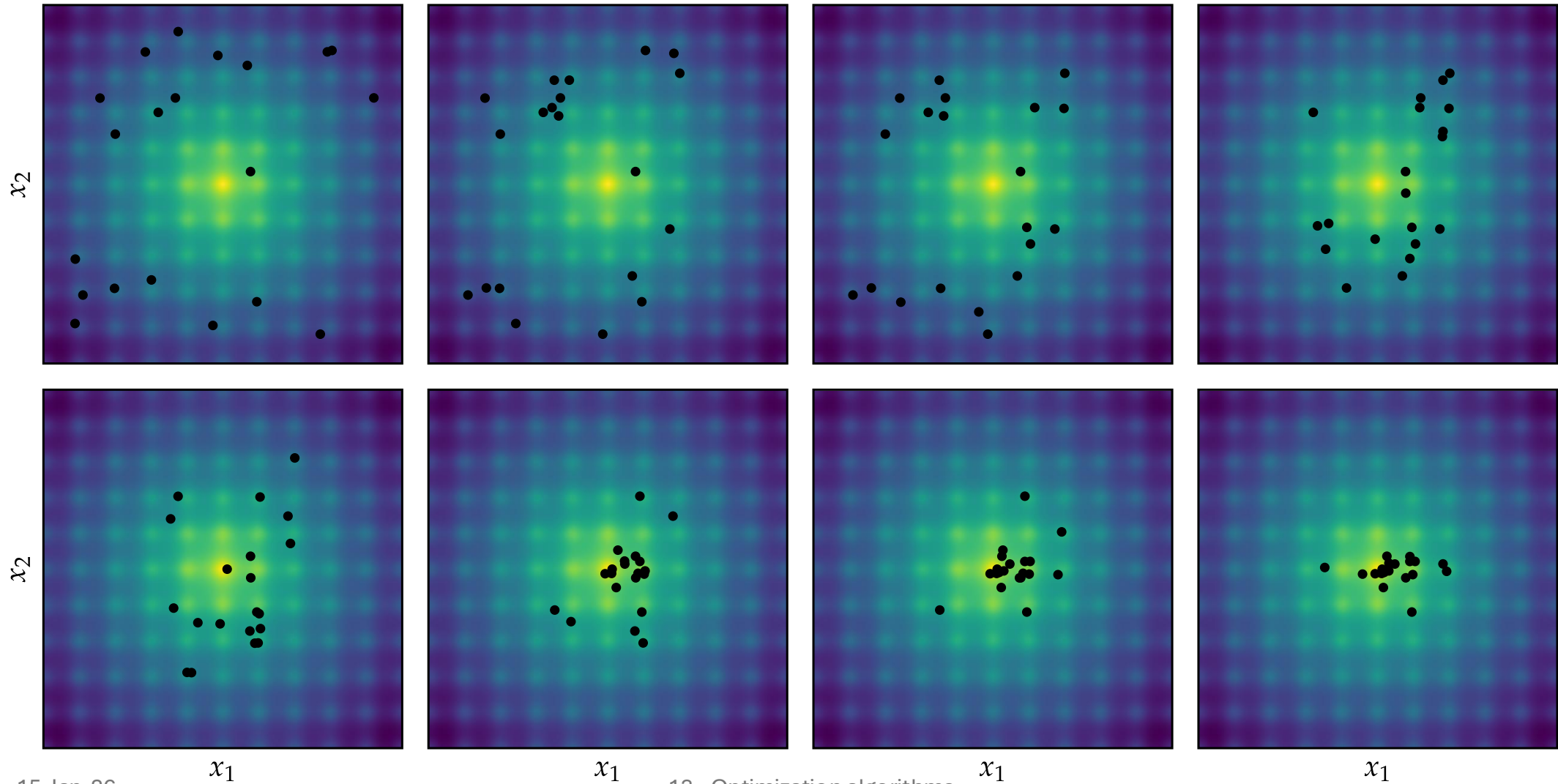
init!(M::DifferentialEvolution, f, designs) = designs

function step!(M::DifferentialEvolution, f, population)
    p, w = M.p, M.w
    n, m = length(population[1]), length(population)
    for x in population
        a, b, c = sample(population, 3, replace=false)
        z = a + w*(b-c)
        x' = crossover(UniformCrossover(p), x, z)
        if f(x') < f(x)
            x .= x'
        end
    end
end

return population
end

```

Differential Evolution



Particle Swarm Optimization

- Each individual, or particle, tracks the following
 - Current position
 - Current velocity
 - Best position seen so far by the particle
 - Best position seen so far by any particle
- At each iteration, these factors produce “force” and “momentum” effects to determine each particle’s movement

$$x^{(i)} \leftarrow x^{(i)} + v^{(i)}$$

$$v^{(i)} \leftarrow wv^{(i)} + c_1r_1\left(x_{\text{best}}^{(i)} - x^{(i)}\right) + c_2r_2\left(x_{\text{best}} - x^{(i)}\right)$$

```

mutable struct Particle
    x      # position
    v      # velocity
    x_best # best design thus far
end

mutable struct ParticleSwarm <: PopulationMethod
    w # inertia
    c1 # first momentum coefficient
    c2 # second momentum coefficient
    V # initial particle velocity distribution
    best # best overall design thus far, and its value
end

function init!(M::ParticleSwarm, f, designs)
    population = [Particle(x,rand(M.V),copy(x)) for x in designs]
    best = (x=copy(population[1].x), y=Inf)
    for P in population
        y = f(P.x)
        if y < best.y; best = (x=P.x, y=y); end
    end
    M.best = best
    return population
end

```

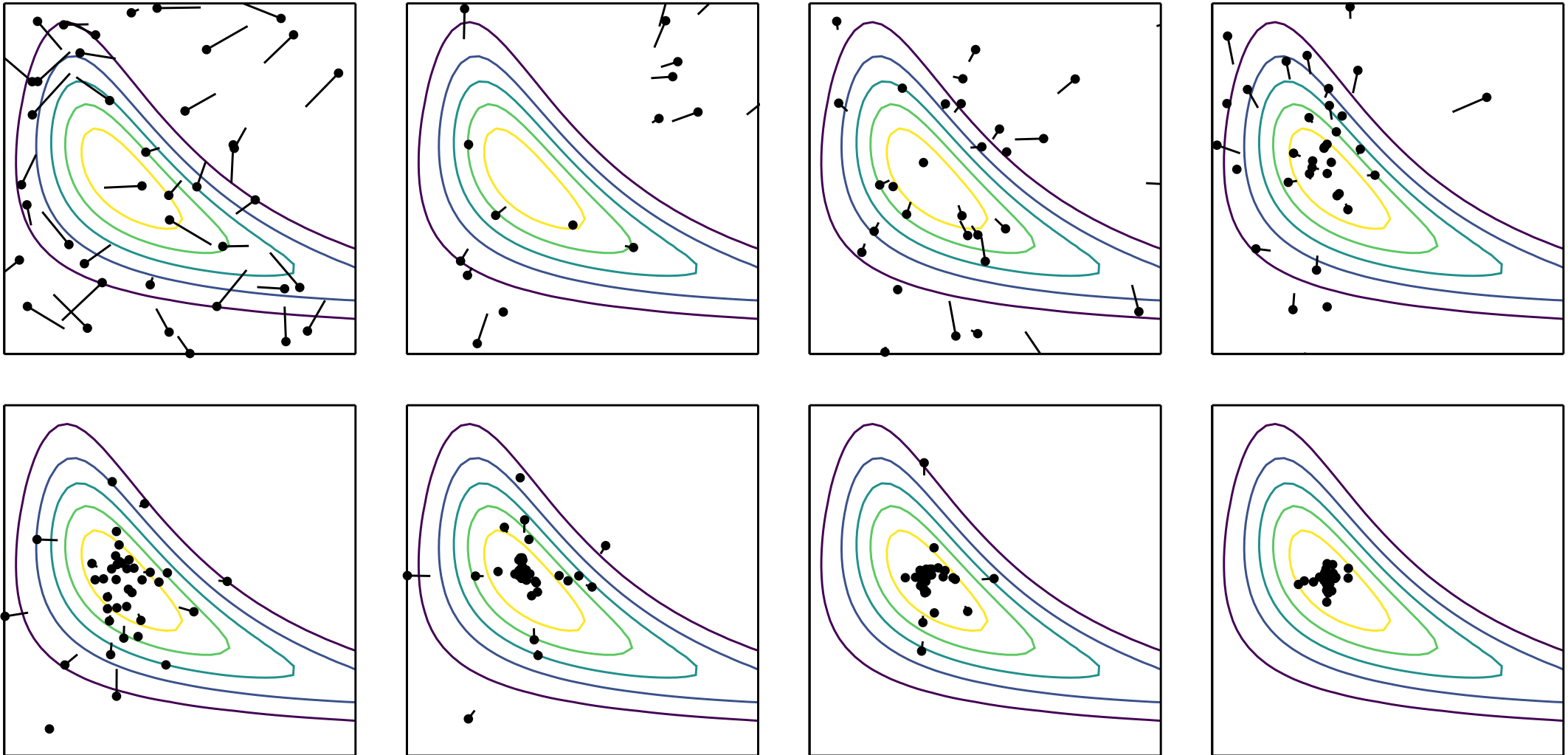
```

function step!(M::ParticleSwarm, f, population)
    w, c1, c2, best = M.w, M.c1, M.c2, M.best
    n = length(best.x)
    for P in population
        r1, r2 = rand(n), rand(n)
        P.x += P.v
        P.v = w*P.v + c1*r1.*(P.x_best - P.x) +
              c2*r2.*(best.x - P.x)

        y = f(P.x)
        if y < best.y; best = (x=copy(P.x), y=y); end
        if y < f(P.x_best); P.x_best .= P.x; end
    end
    M.best = best
    return population
end

```

Particle Swarm Optimization

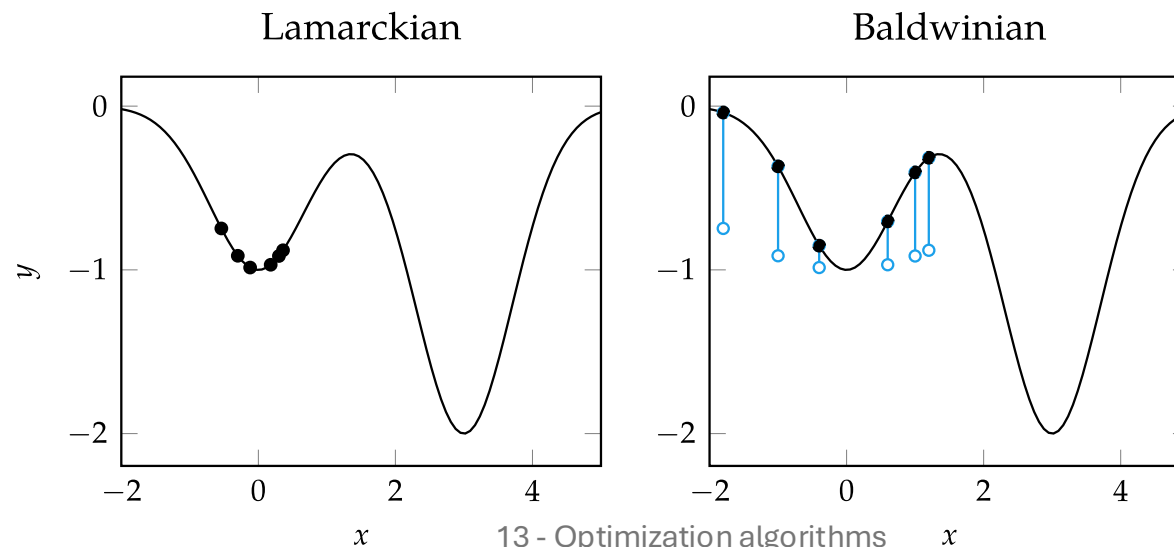


Hybrid Methods

- Generally, population methods are good at finding the best regions in design space, but do not perform as well as descent methods near the minimizer
- Hybrid methods try to leverage the strength of both methods
- Two hybrid approaches
 - Lamarckian learning
 - Baldwinian learning

Hybrid Methods

- Lamarckian learning
 - Performs regular descent method update on each individual
- Baldwinian learning
 - Uses value of descent method update to augment the objective value of each design point



Summary

- Population methods use a collection of individuals in the design space to guide progression toward an optimum
- Genetic algorithms leverage selection, crossover, and mutations to produce better subsequent generations
- Differential evolution, particle swarm optimization, the firefly algorithm, and cuckoo search include rules and mechanisms for attracting design points to the best individuals in the population while maintaining suitable state space exploration
- Population methods can be extended with local search approaches to improve convergence

Summary

- Population methods can be extended to produce individuals that span the Pareto frontier
- Knowing the preferences of experts between pairs of points in the criterion space can help guide the inference of a scalar-valued objective function

Multiobjective Optimization

Multiobjective Optimization

- Some optimization problems have multiple objectives that required tradeoff decisions

Pareto Optimality

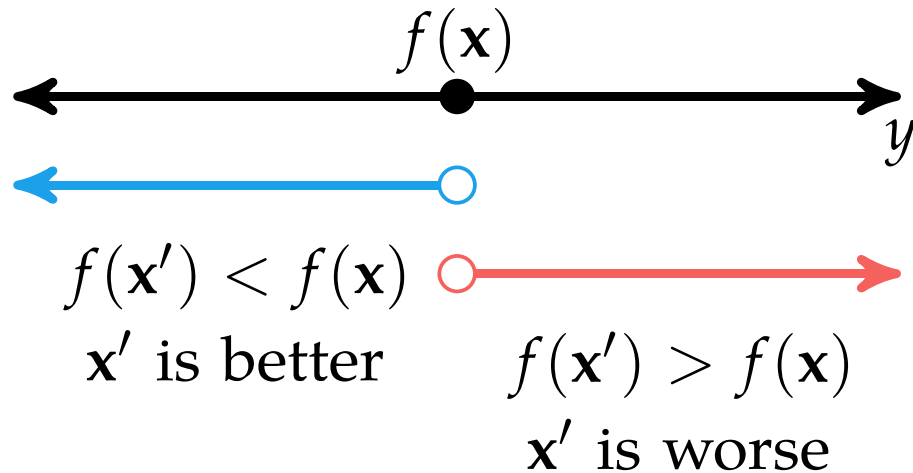
- At a Pareto optimal point, increasing one objective value decreases another
- In multiobjective optimization, one point dominates another if

$$f_i(\mathbf{x}) \leq f_i(\mathbf{x}') \text{ for all } i \text{ in } \{1, \dots, m\}$$

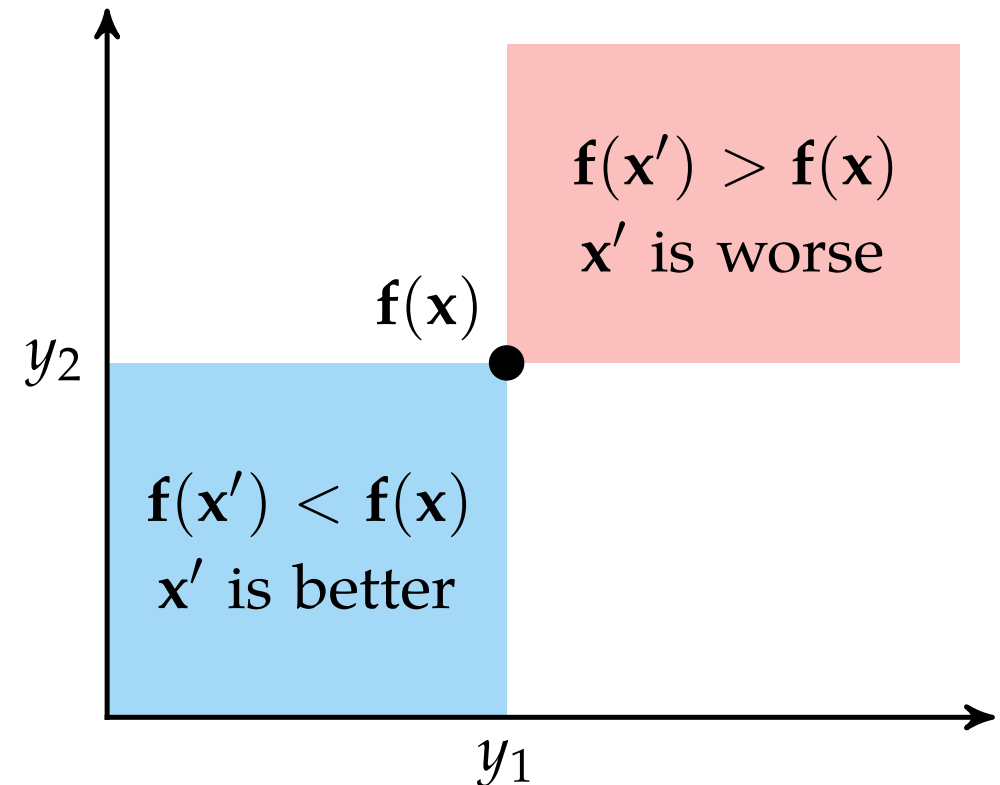
and $f_i(\mathbf{x}) < f_i(\mathbf{x}') \text{ for some } i$

Pareto Optimality

Single Objective

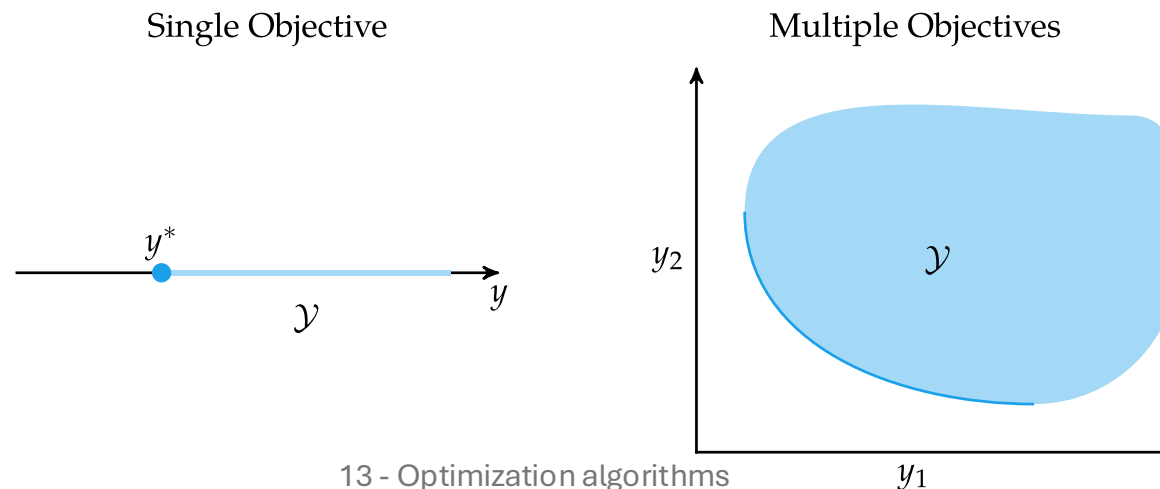


Multiple Objectives



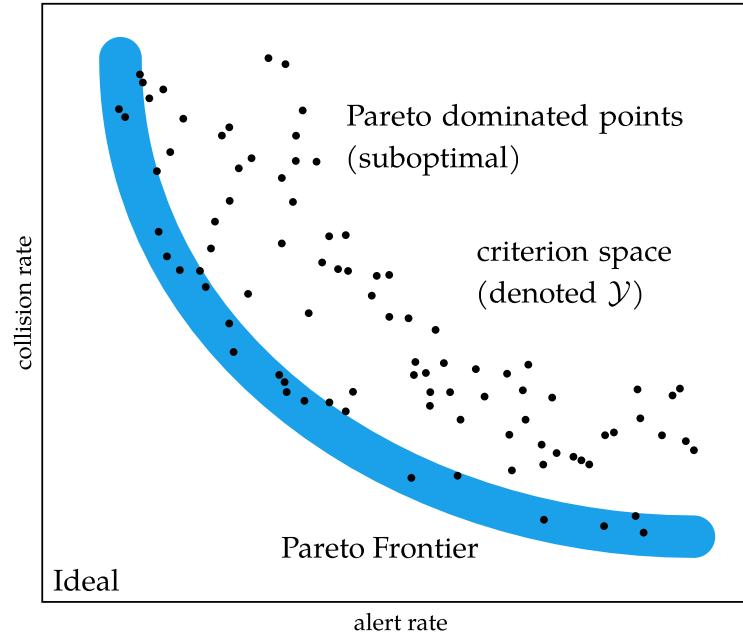
Pareto Optimality: Pareto Frontier

- A criterion space is a set of all outputs possible given a space of inputs
- In a multidimensional optimization problem, the criterion space is m-dimensional.
- In m-dimensional space, it is possible for there to be no clearly optimal point



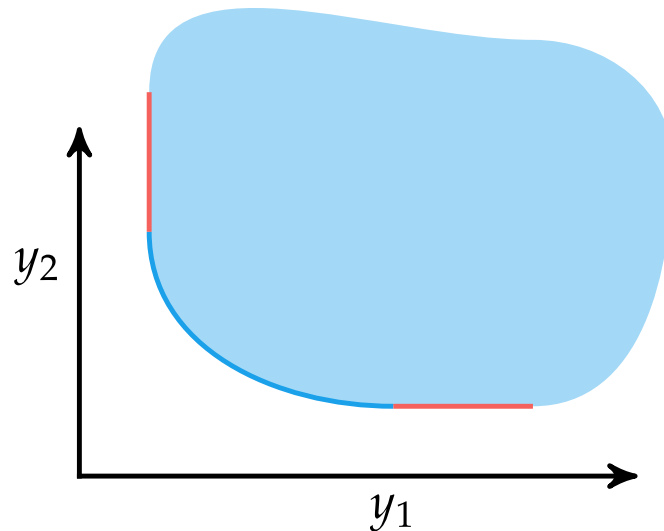
Pareto Optimality: Pareto Frontier

- A design point is considered Pareto-optimal if no other point in the criterion space dominates it
- The set of all Pareto-optimal points forms the Pareto frontier



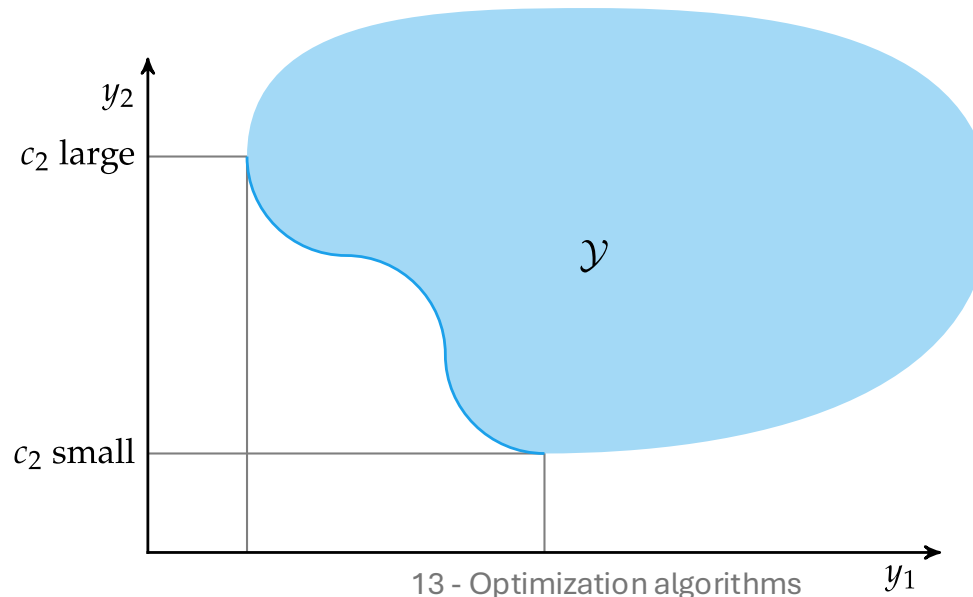
Pareto Optimality: Pareto Frontier

- The utopia point is the component-wise optimal point. It may not be in the actual criterion space
- Weakly Pareto-optimal points are those on the Pareto frontier that cannot be improved simultaneously in all objectives



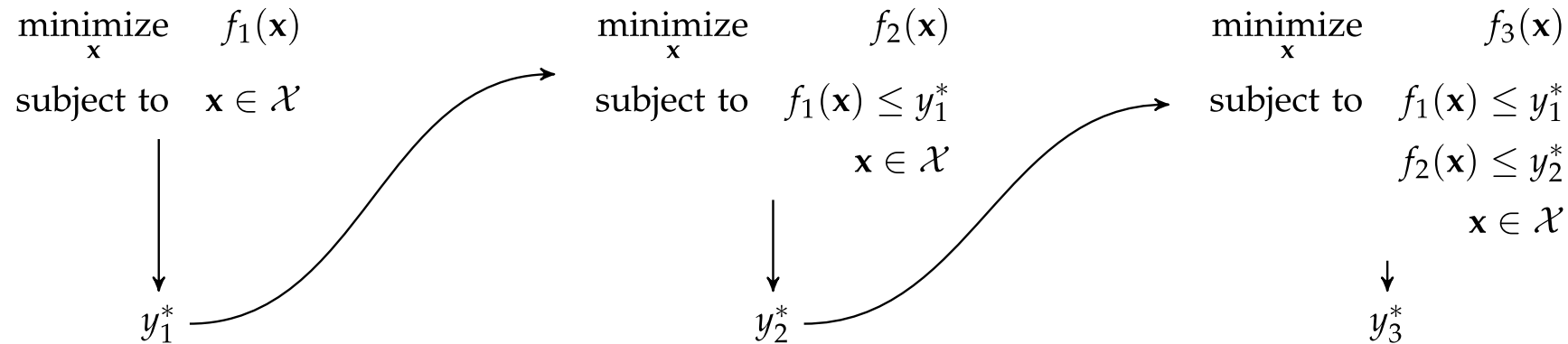
Constraint Methods

- There are various methods for generating the Pareto frontier
- The constraint method constrains all but one of the objectives to produce a unique optimal point in the criterion space
- The constraints are then modified to trace out the Pareto frontier



Constraint Methods

- The lexicographic method ranks the objectives in order of importance and performs a series of single-objective optimizations in order of importance
- Each optimization problem includes constraints to preserve the optimality of the previously optimized objectives



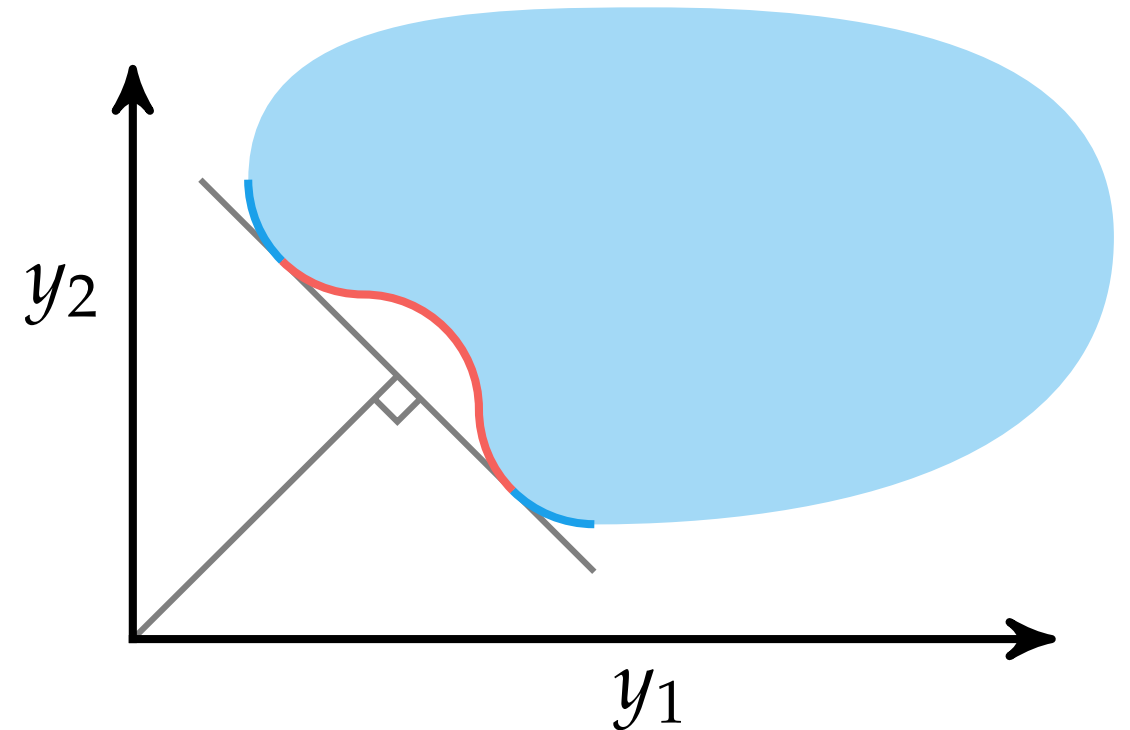
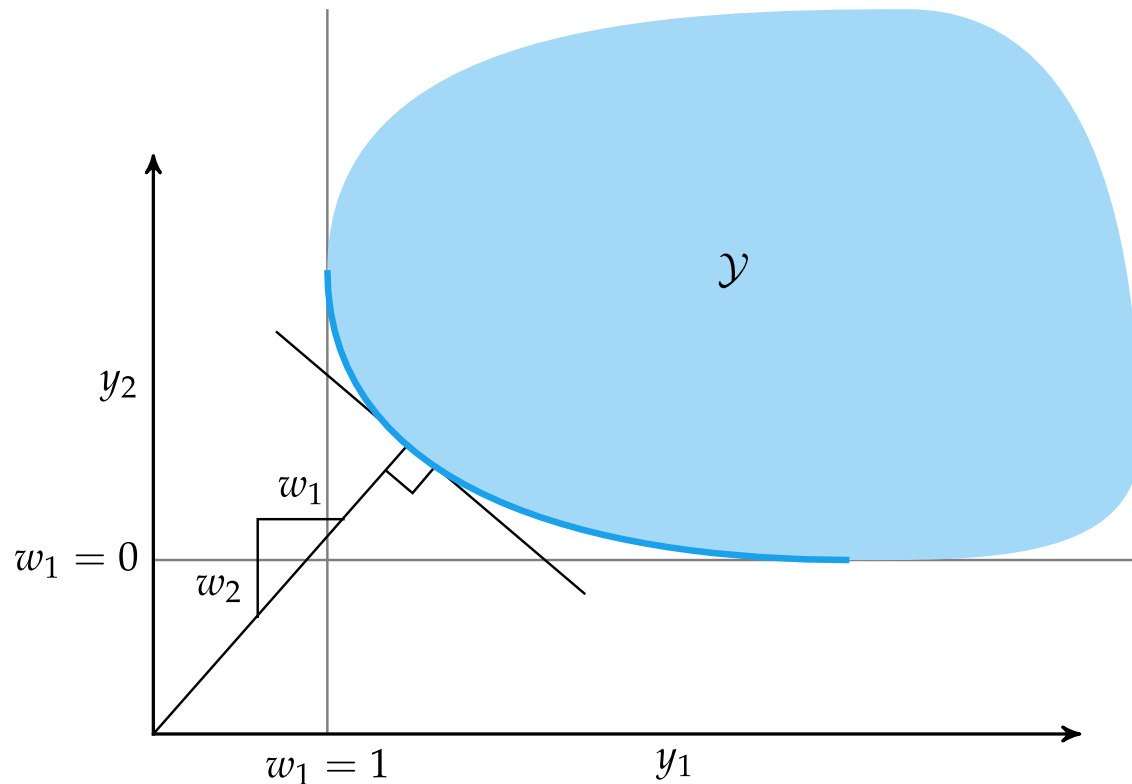
Weight Methods

- A designer can encode preferences over the objectives as a vector of weights
- The Pareto frontier can be generated by sweeping over the space of possible weights
- The weighted sum method combines multiple objective functions into one using a constant vector inner product

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{f}(\mathbf{x})$$

Weight Methods

- Note: weighted sum methods cannot obtain points in red



Weight Methods

- Goal programming converts multiple objectives into a single objective using a norm between $\mathbf{f}(\mathbf{x})$ and a goal point

$$\underset{\mathbf{x} \in \mathcal{X}}{\text{minimize}} \left\| \mathbf{f}(\mathbf{x}) - \mathbf{y}^{\text{goal}} \right\|_p$$

- Typically, the goal point is the utopia point

Weight Methods

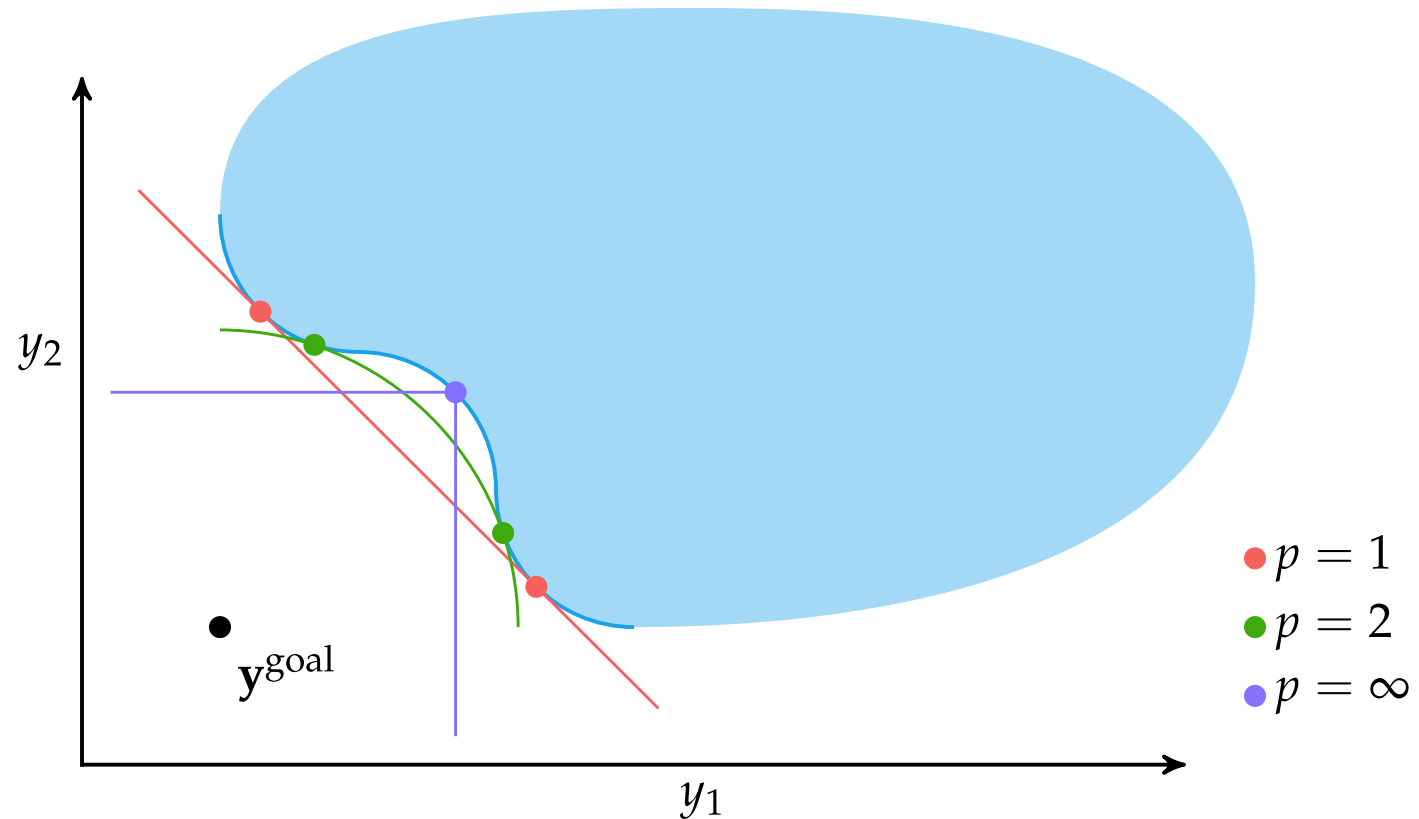
- Weighted exponential sum combines goal programming and the weighted sum method

$$f(\mathbf{x}) = \sum_{i=1}^m w_i \left(f_i(\mathbf{x}) - y_i^{\text{goal}} \right)^p$$

- Here, $\mathbf{w}$ is a vector of weights that sum to 1 and $p \geq 1$

Weight Methods

- Weighted exponential sum for different values of p



Weight Methods

- Previous methods cannot obtain points in nonconvex portions of the Pareto frontier
- The exponential weighted criterion can provide Pareto-optimal points in nonconvex regions
- Note: High values of p can lead to numerical overflow

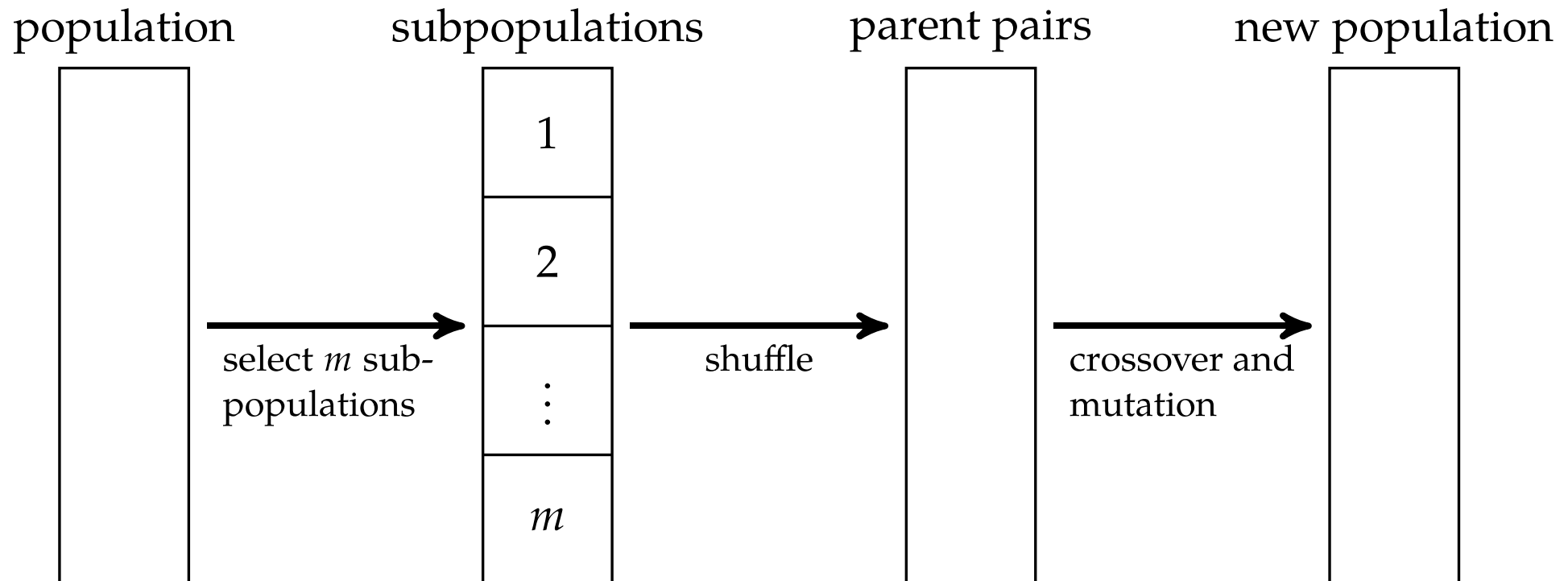
$$f(\mathbf{x}) = \sum_{i=1}^m (e^{pw_i} - 1) e^{pf_i(\mathbf{x})}$$

Multiobjective Population Methods

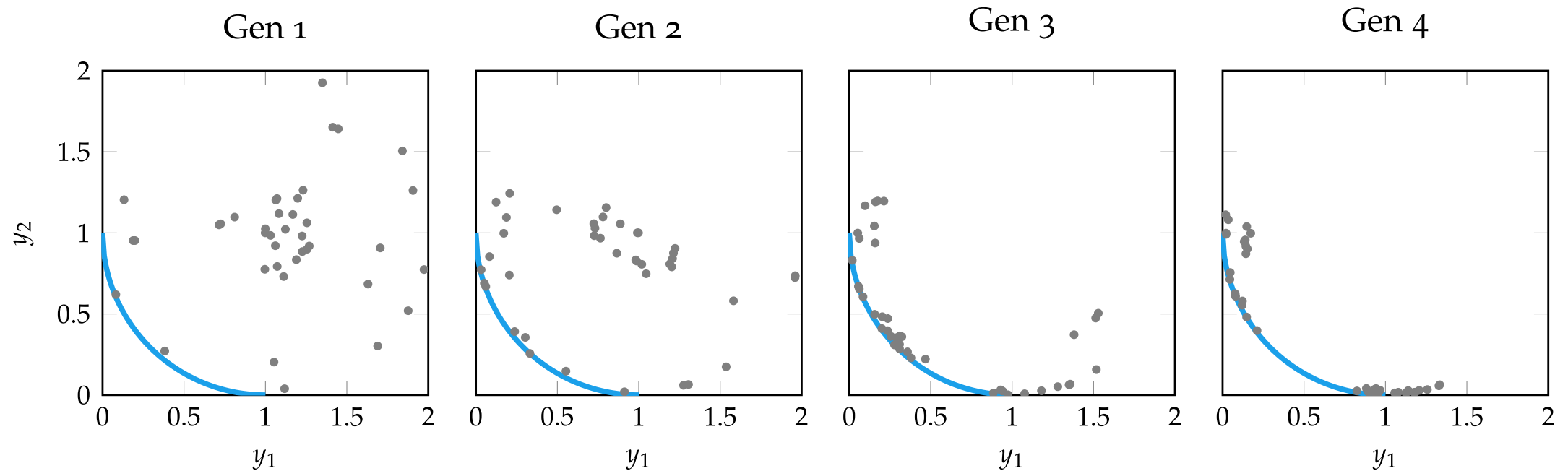
- A population is partitioned into several subpopulations
- Each subpopulation is optimized with respect to a different objective

Multiobjective Population Methods

- The vector evaluated genetic algorithm randomly partitions subpopulations at each generation



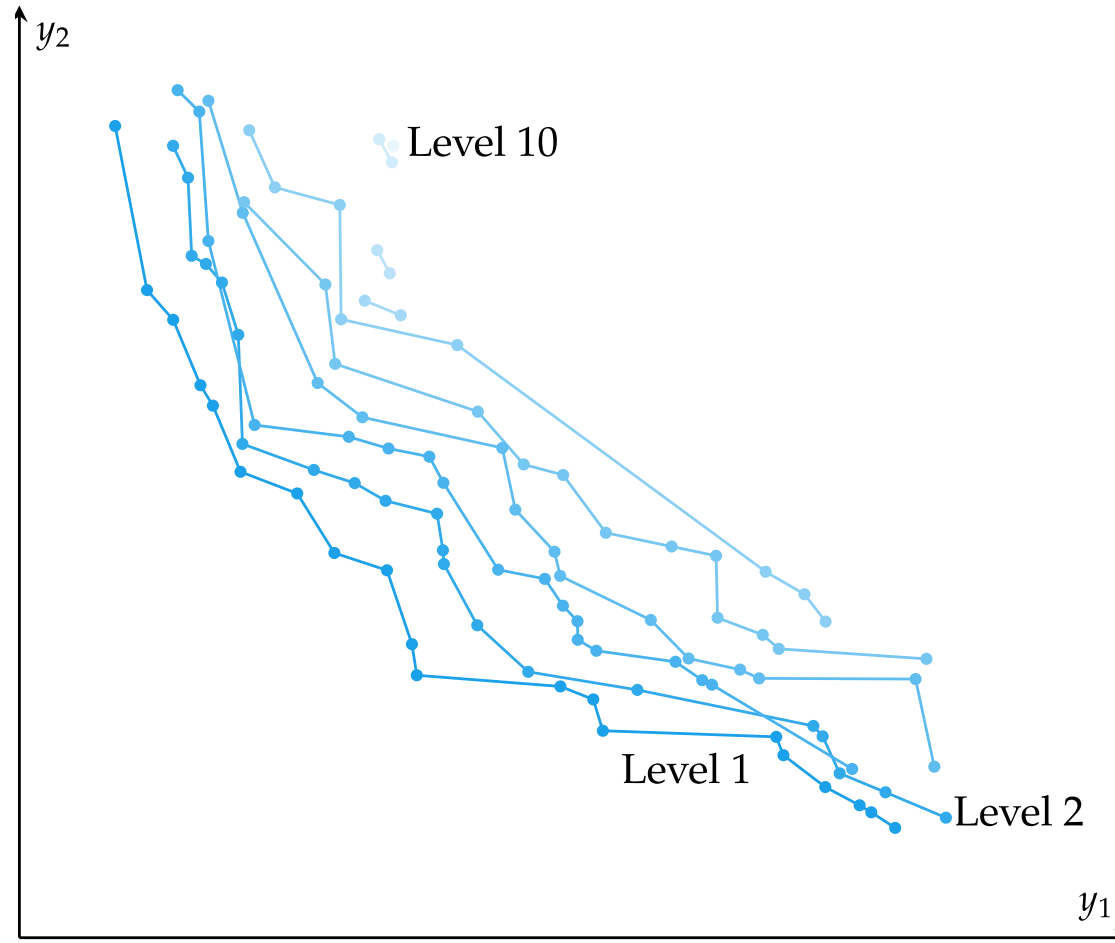
Multiobjective Population Methods



Multiobjective Population Methods

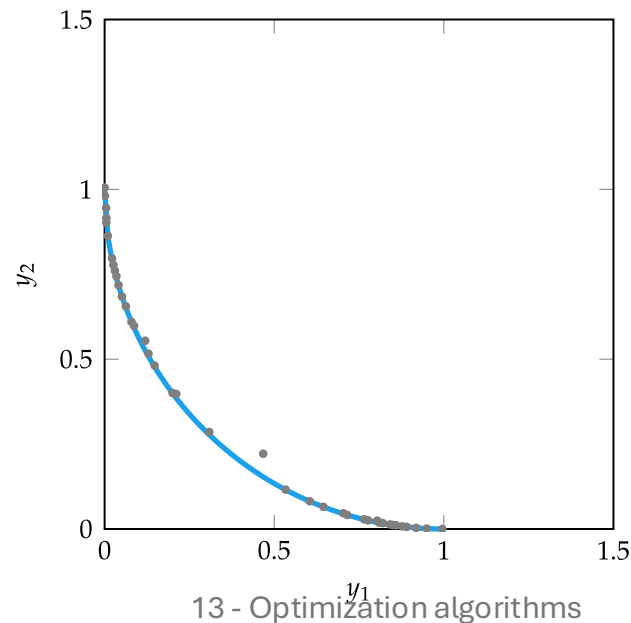
- Nondomination ranking is used to rank individuals in the population
 1. Nondominated individuals (Pareto-frontier)
 2. Nondominated by any except those in (1)
 3. Nondominated by and except those in (1) or (2)
- ...

Multiobjective Population Methods



Multiobjective Population Methods

- In multiobjective population methods, a population that approximates the Pareto frontier is called a Pareto filter
- At every generation, dominant individuals are retained and dominated individuals are removed

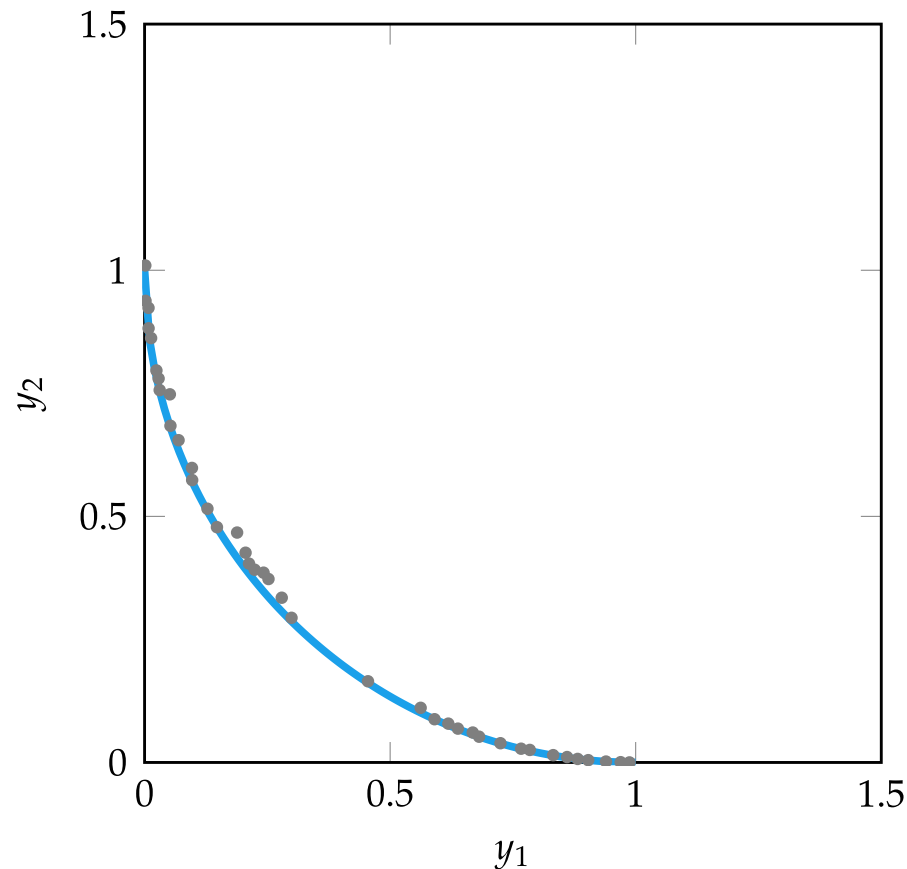


Multiobjective Population Methods

- A niche is a focused cluster of points, typically in the criterion space
- Niche techniques help encourage an even spread of points across the Pareto frontier
- In fitness sharing, the individual objective values are penalized if neighboring points are too close
- In equivalence class sharing, when two individuals are compared, their nondomination ranking is considered first, then fitness sharing is used to as a tie-breaker

Multiobjective Population Methods

- Improved coverage of Pareto frontier using fitness sharing



Summary

- Design problems with multiple objectives often involve trading performance between different objectives
- The Pareto frontier represents the set of potentially optimal solutions
- Vector-valued objective functions can be converted to scalar-valued objective functions using constraint-based or weight-based methods

Expression Optimization

Expression Optimization

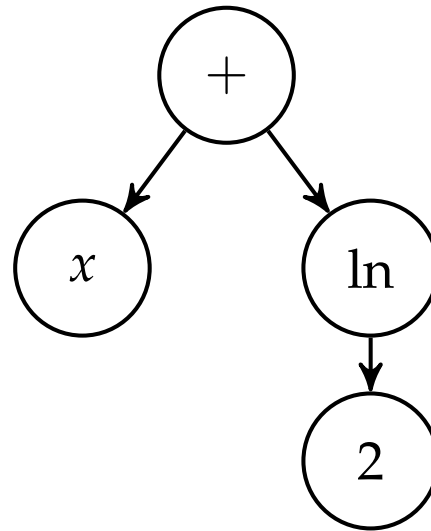
- In some problems, the number of variables is unknown
- Finding optimal solutions of problems with unknown parameters requires optimizing over the set of expressions

Grammars

- Expressions can be represented by a tree of symbols
- Grammars constrain the space of possible expressions
- A grammar is represented by a set of production rules
- Production rules involve symbols and types
- Types are defined sets of expression trees
- Production rules are used to form expression trees by expanding types into new expressions with more types and symbols
- If a type is expanded into just symbols, preventing further expansion, then it is called a terminal

Grammars

Representing the expression $x + \ln 2$ as a tree



Grammars

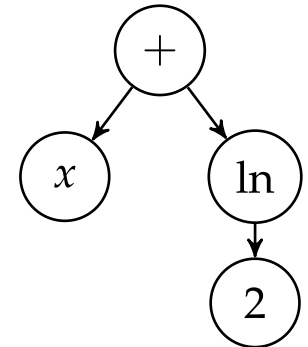
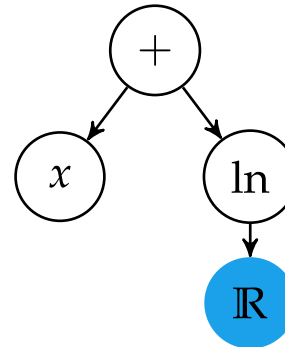
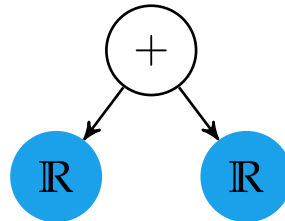
Constructing the expression tree using a grammar

$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$$

$$\mathbb{R} \mapsto x$$

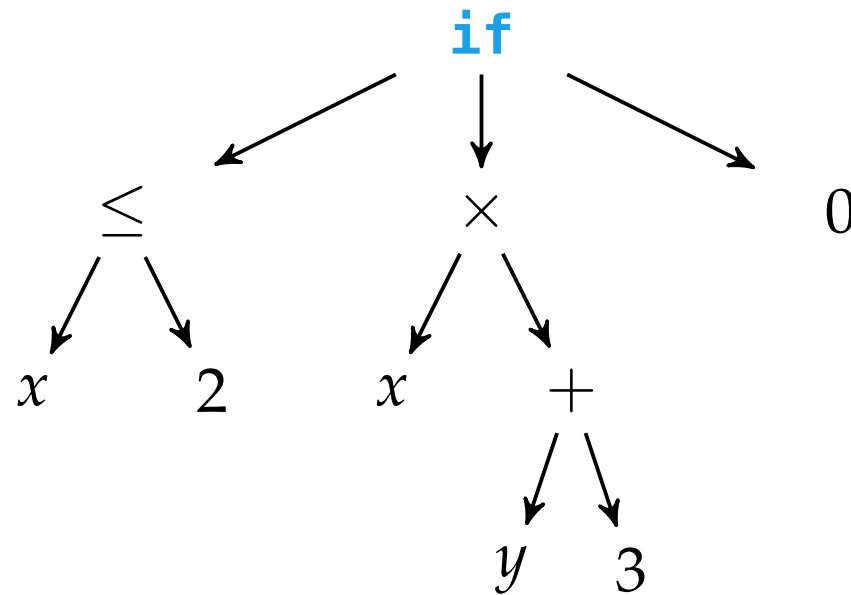
$$\mathbb{R} \mapsto \ln(\mathbb{R})$$

$$\mathbb{R} \mapsto 2$$



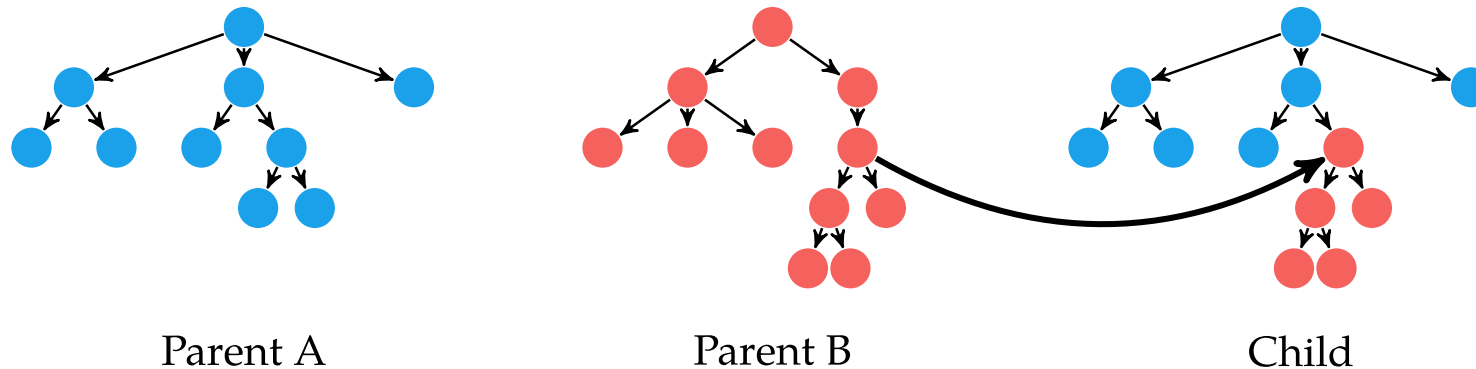
Genetic Programming

- Genetic programming represents individuals as trees and optimizes over them using mutation and tree crossover



Genetic Programming

- Tree crossover combines two parent trees into a child tree



- Over many generations, tree crossover tends to increase tree depth and overall complexity
- It is common to introduce a bias for parsimony, or simplicity in the solution based on the tree's depth

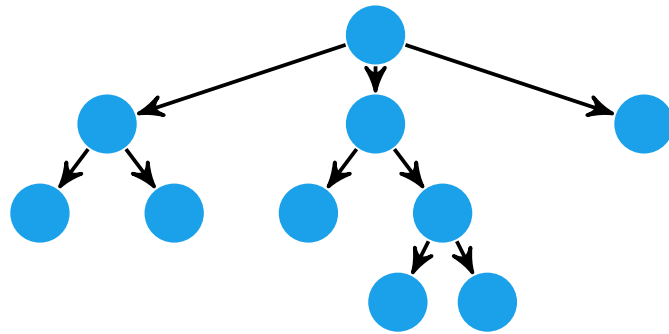
```

struct TreeCrossover <: CrossoverMethod
    grammar
    max_depth
end
function crossover(C::TreeCrossover, a, b)
    child = deepcopy(a)
    crosspoint = sample(b)
    typ = return_type(C.grammar, crosspoint.ind)
    d_subtree = depth(crosspoint)
    d_max = C.max_depth + 1 - d_subtree
    if d_max > 0 && contains_returntype(child, C.grammar, typ, d_max)
        loc = sample(NodeLoc, child, typ, C.grammar, d_max)
        insert!(child, loc, deepcopy(crosspoint))
    end
    child
end

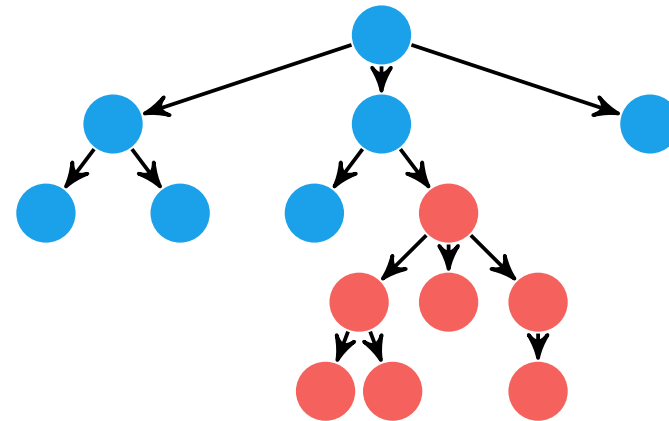
```

Genetic Programming

- Random variation can be added in multiple ways
- Tree mutation is when a random tree is generated and its root replaces a random node in the tree



Before



After

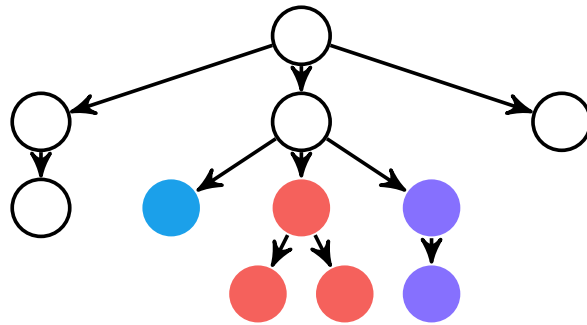
```

struct TreeMutation <: MutationMethod
    grammar
    p
end
function mutate(M::TreeMutation, a)
    child = deepcopy(a)
    if rand() < M.p
        loc = sample(NodeLoc, child)
        typ = return_type(M.grammar, get(child, loc).ind)
        subtree = rand(RuleNode, M.grammar, typ)
        insert!(child, loc, subtree)
    end
    return child
end

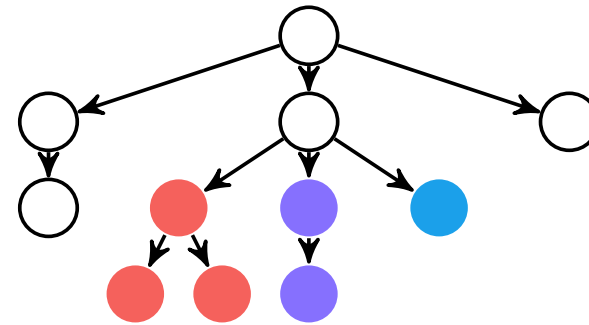
```

Genetic Programming

- Tree permutation is when randomly chosen nodes are permuted
- Often combined with tree mutation



Before



After

```

struct TreePermutation <: MutationMethod
    grammar
    p
end
function mutate(M::TreePermutation, a)
    child = deepcopy(a)
    if rand() < M.p
        node = sample(child)
        n = length(node.children)
        types = child_types(M.grammar, node)
        for i in 1 : n-1
            c = 1
            for k in i+1 : n
                if types[k] == types[i] &&
                    rand() < 1/(c+=1)

                    node.children[i], node.children[k] =
                        node.children[k], node.children[i]

                end
            end
        end
    end
    return child
end

```

Summary

- Optimization is the process of finding the best system design subject to a set of constraints
- Optimization is concerned with finding **global** minima of a function
- Minima can occur where the gradient is zero, but zero-gradient does not imply optimality